

From Bayesian Optimization to an Agentic, Claude-Driven Tuning Loop

Skipper-CCD operating-point optimization with an LTA readout

Juan Cruz Estrada Vigil

October 1, 2026

Outline

1. The starting point: Bayesian optimization of a Skipper-CCD
2. Code review: what to watch for
3. Agentic version: three cooperating agents
4. Claude as the optimizer
5. Running it in the lab: what went wrong and how it was fixed
6. Cost
7. Lab notebook: memory across campaigns
8. GP vs. Claude, open questions, next steps

The problem

- ▶ Skipper-CCD read out with an **LTA** controller
- ▶ Find the operating parameters that give the best images
- ▶ Each evaluation is a **real measurement**:
expose (LED) → set parameters → read out → score
- ▶ Expensive, noisy, black-box ⇒ Bayesian optimization

Parameter	Range	
V_{dd}	$[-23, -10]$	V
V_{drain}	$[-23, -10]$	V
V_r	$[-8, -5]$	V
delay_H_overlap	$[10, 30]$	clk

Optimizer works in $[-1, 1]^4$; values mapped to physical units.

The objective function

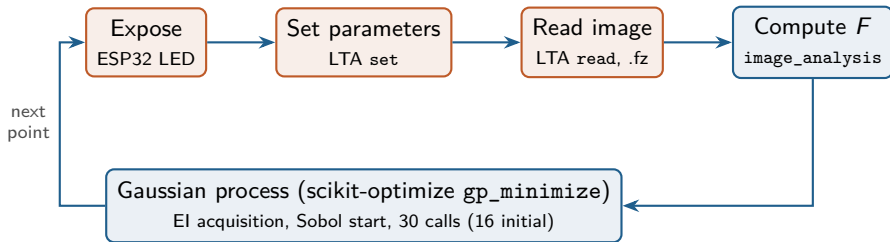
From each image: active region and overscan region

$$\text{gain} = \text{median}(\text{active}) - \text{median}(\text{overscan}), \quad \sigma_{os} = \text{std}(\text{overscan})$$

$$F = \min\left(\left|\frac{\sigma_{os}}{\text{gain}}\right|, 1000\right) + \sum_i c_i \text{penalty}_i^{p_i} \quad (\text{lower is better})$$

- ▶ With a fixed LED exposure, $\text{gain} \propto \text{ADU}/e^-$, so F tracks the read noise in electrons
- ▶ Optional penalties (negative gain, saturation, ...) are configurable; all off in the current campaign

The original loop



- ▶ Modular: `lta_control` (hardware), `image_analysis` (FITS + F), `bo_core` (optimizer, CSV, plots)
- ▶ One JSON config per campaign; warm restart from a previous run

Code review: things to watch

- ▶ **Stale images:** shell return codes ignored and the newest .fz is scored, so a failed readout would be scored using the previous image
- ▶ **Bad amplifier states can score well:** `abs()` hides an inverted gain; a clipped output has $\sigma_{os} \rightarrow 0$
- ▶ **Real example** from a lab run:

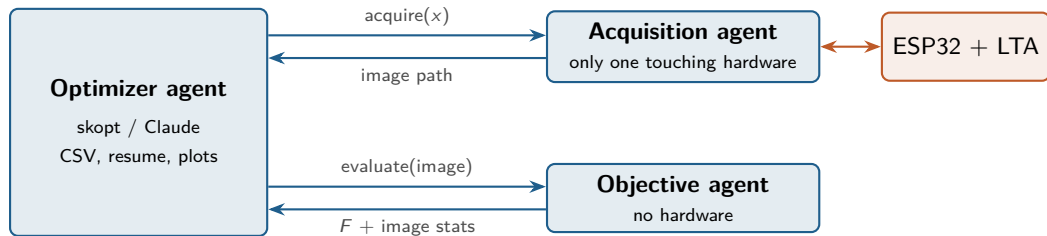
V_{dd}	V_{drain}	V_r	delay	σ_{os}	gain	F
-10.3	-12.5	-5.2	15	31924	-3544	9.0

a broken output still gets a finite, “reasonable” score

- ▶ **No settling time** after changing bias voltages
- ▶ **Noisy F :** the best *measured* point may be a lucky draw
- ▶ **Resume file** only written at the end of a run

Kept as-is on purpose (no functional changes); the resume issue was fixed later.

Step 1: three cooperating agents



- ▶ Each agent is its own process (Python multiprocessing, standard library only)
- ▶ Messages are plain JSON dicts: ready for sockets or an LLM later
- ▶ Every message is logged to `<run>_messages.jsonl`
- ▶ Same command line, same config, same output files

Messages between agents

```
{"id": 7, "kind": "acquire", "sender": "optimizer",  
  "payload": {"iteration": 3, "x_norm": [0.12, -0.40, 0.75, -0.05]}}  
  
{"id": 8, "kind": "reply", "sender": "acquisition", "reply_to": 7, "ok": true,  
  "payload": {"image_path": ".../optimize_76.fz"}}  
  
{"id": 9, "kind": "evaluate", "sender": "optimizer",  
  "payload": {"iteration": 3, "image_path": ".../optimize_76.fz"}}  
  
{"id": 10, "kind": "reply", "sender": "objective", "reply_to": 9, "ok": true,  
  "payload": {"F": 0.0296, "stats": {"noise_overscan": 636.4,  
    "charge_overscan": 5310.0, "charge_active": 26812.5, "gain": 21502.5}}}}
```

- An agent error comes back as "ok": false with its traceback; the run stops cleanly and the other agents are shut down

Validation: same results as the original

Simulated hardware for testing:

- ▶ fake `lta.sh`: synthetic images whose noise and signal depend on the voltages set
- ▶ fake ESP32 and FITS reader

Both drivers, same config and seed:

- ▶ identical `gp_results.csv`, progress CSV and image files
- ▶ identical results on warm restart
- ▶ agent failure → clean stop with the agent's error

Test suite

17 automated tests, all passing
equivalence, message log, resume, failures,
Claude optimizer (with a fake Claude),
retries, cost, notebook

Runs without hardware, network or API key:

```
python -m pytest tests
```

First lab deployment: one surprise

- ▶ Error: “*No .fz image found*”, but the LTA said “*Read done*”
- ▶ Cause: relative output path `images/...`
 - ▶ Python resolved it from *its* working directory
 - ▶ the LTA daemon resolved it from *its own* directory
 - ▶ $\Rightarrow$ images written to one folder, searched for in another
- ▶ Fix: absolute `output_base` in the config (no code change)
- ▶ Not specific to agents: the original script had the same latent issue

Step 2: Claude replaces the Gaussian process

1. **Sobol start:** 8 space-filling points (as for the GP)
2. **Each further point:** Claude gets
 - ▶ parameter descriptions and bounds, objective definition
 - ▶ *all* measurements so far, in physical units
 - ▶ image statistics (σ_{os} , σ_{act} , charges, gain), which the GP never sees
 - ▶ operator notes and the lab notebook
3. Claude returns the next point as **structured JSON**; values clipped to bounds

Drop-in replacement

`claude_minimize()` has the same interface and returns the same result object as `gp_minimize()`: progress CSV, resume, pickle and convergence plot unchanged.

```
"optimizer": {  
  "type": "claude",  
  "n_calls": 30,  
  "n_initial_points": 8,  
  "model": "claude-opus-5-5",  
  "effort": "high",  
  "notes": "..."} }
```

What Claude sees on each turn

Measurements so far (16):

#	Vdd	Vdrain	Vr	delay_H_overlap	F	noise_overscan	...	gain
1	-16.5	-16.5	-6.5	20	0.071	410.2	...	5790
...								
8	-10.3	-12.5	-5.2	15	9.007	3.19e+04	...	-3544
...								
16	...	0.0296	636.4	...	2.15e+04			

Measurements left in this campaign, including the next one: 14.
Choose the next point.

Each iteration prints and logs:

- ▶ Claude proposes: Vdd=..., Vdrain=..., Vr=..., delay_H_overlap=...
- ▶ a **summary of Claude's thinking** (why this point)
- ▶ tokens used and the estimated cost

Table layout as sent to Claude. Values in rows 1 and 16 are illustrative; row 8 shows the statistics of the broken lab image.

Running it in the lab: issues and fixes

Symptom	Cause	Fix
401 invalid x-api-key	placeholder, then a model ID, used as the key	real key from the Console; instructions clarified
Request declined: <i>reasoning_extraction</i>	prompt asked Claude to write out its reasoning in the answer	ask only for the next point; read Claude's <i>thinking summary</i> from the API
529 Overloaded after 16 images	temporary API load; SDK retried twice, then the run stopped	retries with growing waits (up to ~15 min)
16 images could not be resumed	gp_results.csv written only at the end	resume file written after every image

Lesson: with real hardware, the **surrounding software** (paths, keys, retries, checkpoints) matters as much as the optimizer.

Results so far

First Claude campaigns (both interrupted):

- ▶ run stopped at 16 images (API overload):
best $F = 0.0296$ (optimize_76.fz),
 $\sigma_{os} = 636$ ADU, gain = 21 500 ADU
- ▶ earlier run stopped at 8 images (API key):
best $F = 1.47$; broken output at
 $V_{dd} = -10.3$ V

To do:

- ▶ complete campaign; GP vs. Claude on the same day
- ▶ re-measure best points (noise of F)

[FILL IN: final numbers]

Convergence plot
copy results-skipper.pdf to
figures/convergence.pdf

Cost

Claude Opus 5.5 (list price)	per M tokens
input	\$4.00
output (incl. thinking)	\$20.00
cached input, read	\$0.20

Per campaign (30 calls, 8 Sobol):

- ▶ 22 decision requests + 1 notebook entry
- ▶ fixed instructions cached → cheap repeats
- ▶ measured: **[FILL IN from Console]**

Tracked automatically:

- ▶ each iteration: tokens, cost, running total
- ▶ end of run: summary
- ▶ saved in
`<run>_claude_decisions.jsonl` and
the result pickle
- ▶ Console Usage / Cost pages are the billed truth

Perspective

API cost per campaign is small compared to lab time per image; every image the optimizer saves is worth far more.

Lab notebook: memory across campaigns

- ▶ Each API call starts from scratch: no learning between runs
- ▶ **End of campaign:** Claude writes a short entry (best region, broken settings, sensitivities, suggestions)
- ▶ **Start of next campaign:** notebook loaded into Claude's instructions as *prior knowledge*
- ▶ Current measurements take priority over old entries
- ▶ Plain Markdown file: operators read, edit, add their own notes

```
# Claude lab notebook

## 2026-10-01 21:40 | module skipper |
  30 measurements (22 new) |
  best F = 0.0296 at Vdd=..., Vr=...

- Best region: ...
- Broken: Vdd above ~ -11 V
  (negative gain, sigma_os > 30000 ADU)
- Most sensitive to: ...
- Next time: ...
```

Header written by the code (facts); text by Claude. Illustrative entry.

Gaussian process vs. Claude

	Gaussian process	Claude
Information used	parameters and F only	parameters, F , image statistics, notes, notebook
Reproducible	yes (fixed seed)	no (choices can vary)
Explains choices	no	yes (thinking summary per point)
Uncertainty model	explicit (GP posterior)	implicit
Memory across runs	warm restart (data only)	data + written lessons
Cost	free	small API cost per campaign
Failure modes	mis-fit on outliers	API availability, safety filters

Same agents, same outputs: switching is one line in the config.

Open questions and next steps

- ▶ **Noise of F :** repeat one point ~ 10 times; needed for any convergence or “which point is better” statement
- ▶ **Stopping rule:** today a fixed budget of `n_calls`; options: no-improvement rule, or Claude declares convergence
- ▶ **Head-to-head:** GP vs. Claude, same day, same budget
- ▶ **Better objective:** ask Claude (with literature search) to define “*best flat field for astronomy*” as computable criteria, reviewed by us, then computed deterministically
 - ▶ may need flat pairs / several exposure levels per iteration
- ▶ **Robustness:** API key check at start-up, flag broken images (negative gain), cheaper models (Haiku)

Summary

1. Reviewed the existing Bayesian optimization of the Skipper-CCD
2. Rebuilt it as three agents (acquisition, objective, optimizer), with results verified identical to the original
3. Added Claude as a drop-in optimizer that uses the image statistics and explains every choice
4. Made it lab-robust: absolute paths, retries, per-image checkpoints
5. Added cost tracking and a lab notebook that carries lessons from one campaign to the next

Code: `optimize_agents.py`, `agents/`, `tests/`